

Folien-Analyse — HTML (Säule 1)

Quelle: html/Day1 Tags-Attribute · Day2 Inline-Block-Sonderz.-Listen-Tabellen · Day3 Verweise-Bilder-URL-HTTP-Formulare · Day4
Formulare-Input-Head. Gegen MDN/WHATWG verifiziert. Folienfehler mit . Themen-IDs = Tracker (H1-H8).

H1 — Minimales valides HTML-Dokument

- **Definition:** HTML = Auszeichnungssprache (Markup), beschreibt Struktur von Text (Überschriften, Absätze, Links, Tabellen...). Tags zeichnen aus, kein Layout.
- **Pflichtbestandteile:** Doctype + `<html>` mit `<head>` (Zeichensatz + Titel) und `<body>`.

```
<!DOCTYPE html>           <!-- HTML5-Doctype, kein Versionsattribut -->
<html lang="de">
<head>
  <meta charset="UTF-8">    <!-- PFLICHT: Zeichensatzdeklaration -->
  <title>Titel</title>      <!-- PFLICHT: genau ein Titel -->
</head>
<body>
  <h1>Inhalt</h1>
</body>
</html>
```

Tag-Regeln (Day1)

Diese Regeln bestimmen, wie Tags korrekt geschrieben und ineinander verschachtelt werden - die Grundlage jeder HTML-Produktionsaufgabe.

Regel	Inhalt
Struktur	<code><tag>...</tag></code> , öffnend + schließend
Schachtelung	RICHTIG <code><a></code> ; FALSCH <code><a></code> (Überlappung verboten)
Void/Empty Elements	kein Inhalt, KEIN schließendes Tag: <code>
</code> , <code><hr></code> , <code></code> , <code><meta></code>
Attribut	<code><p lang="fr"></code> , abhängig vom Tag; manche global

Fallen / Bugs

- **Falle:** `<head>` muss mind. `<meta charset>` + `<title>` enthalten — typische Produktionsaufgabe ("minimales valides Dokument angeben").
- **Falle:** Anführungszeichen weglassen / wertlose Attribute = laut Folie "ganz schlechter Stil", aber HTML erlaubt es (XML nicht).
- **HTML vs. XML:** HTML weniger strikt — nicht alle Tags müssen geschlossen werden, Quotes optional, case-insensitiv. XML strikt.
- `<hr></hr>` ist NICHT äquivalent erlaubt: bei Void-Elementen ist nur `<hr>` (oder `<hr/>`) korrekt.

Drill

- [Erstellen] Minimal valides HTML5-Dokument mit Titel "Test".

Lösung: siehe Blueprint oben (Doctype, html, head mit charset+title, body).

- [MC] Welche müssen geschlossen werden: `<p>`, `<br>`, `<img>` ?

Lösung: nur `<p>`. `<br>` / `<img>` sind Void-Elemente.

H2 — Block- vs. Inline-Elemente

	Block	Inline
Zeilenumbruch	beginnt neue Zeile + Umbruch danach	kein Umbruch, fließt im Text
darf enthalten	Block + Inline	nur Inline, keine Blockelemente
Abstände	i.d.R. vor/nach	normal keine
Beispiele	h1–h6, p, div, article, section, ul, table, hr, pre, blockquote	b, i, sup, sub, br, em, strong, span, a, code

Fallen / Bugs

- **Falle:** Inline-Element darf KEIN Blockelement umschließen (`<b><p>...</p></b>` ungültig).
- **Hinweis:** Block/Inline ist die *Default-Darstellung*; via CSS `display` überschreibbar (gehört zu CSS C5).

Drill

- [MC] Block oder Inline: `<p>`, `<span>`, `<sup>`, `<section>` ?

Lösung: Block, Inline, Inline, Block.

H3 — `<input>`-Attribute (Produktion/MC)

- **Definition:** `<input>` erzeugt Steuerelemente zur Datenerfassung; `name` = Parametername beim Submit.

Attribut	Funktion	Besonderheit (Klausur)
name	Parametername	ohne name → wird NICHT gesendet
value	Startwert	je nach type
disabled	komplett deaktiviert	kein Fokus, wird NICHT gesendet
readonly	nicht editierbar	fokussierbar, WIRD gesendet
required	Pflichtfeld	leere Eingabe abgelehnt
pattern	RegEx-Format	clientseitige Validierung
placeholder	Hinweistext	verschwindet bei Eingabe (≠ value)
maxlength/minlength	Längengrenzen	positive ganze Zahl
size	sichtbare Zeichen	nur Darstellung
autocomplete/autofocus	Autovervollst./Fokus	
list	+ <code><datalist></code> per id	Vorschlagswerte

Fallen / Bugs

- **Kontrast** `disabled` vs. `readonly` : disabled = kein Fokus + sendet NICHT; readonly = Fokus möglich + sendet. **Häufigste MC-Falle.**
- **Falle:** Nicht-angekreuzte `checkbox` / `radio` senden GAR KEINEN Parameter.
- **Falle:** `placeholder` ist kein Wert — leeres Feld sendet leer.

Drill

- [MC] Welches Feld erscheint im Request-Body: `disabled` oder `readonly` ?

Lösung: `readonly` .

- [Erstellen] Pflicht-Email-Feld namens "mail".

Lösung: `<input type="email" name="mail" required>`

H3b — `<input type="...">` (Typen)

- **Default:** `text`. Werte: `hidden`, `text`, `search`, `tel`, `url`, `email`, `password`, `date`, `datetime-local`, `month`, `week`, `time`, `number`, `range`, `color`, `checkbox`, `radio`, `file`, `submit`, `image`, `reset`, `button`.

type	Verhalten / Besonderheit
text/search	nur stilistischer Unterschied, keine Umbrüche
hidden	unsichtbar, reicht Parameter/Token durch
password	Eingabe maskiert
tel	KEINE Validierung (→ pattern nötig)
email/url	Validierung auf gültige Adresse / absolute URL
number/range	min,max,step
color	Farbauswahl → Hex #rrggbb FOLIE sagt "8-Bit RGB" → korrekt: 24-bit (8 bit pro Kanal)
checkbox	checked; gesendet nur wenn angekreuzt (value oder on)
radio	gleiche name → genau eine Auswahl
file	accept, multiple; braucht <code>enctype="multipart/form-data"</code> + <code>method="post"</code> sonst nur Dateiname
submit	sendet Formular; name+value → mehrere Buttons unterscheidbar
image	Bild-Submit, sendet zusätzlich Klick-Koordinaten
reset	Formular zurücksetzen
button	ohne Funktion, für Scripting

Fallen / Bugs

- FOLIE: `datetime` "praktisch keine Browserunterstützung" → korrekt heute: `datetime` ist deprecated, stattdessen `datetime-local`.
- **Falle:** File-Upload ohne `enctype=multipart/form-data` überträgt nur den Dateinamen.

Drill

- [MC] Welcher type validiert die Eingabe automatisch: `tel` oder `email` ?

Lösung: `email` (tel nicht).

H4 — Formulare (`<form>`)

- **Definition:** `<form>` bündelt Eingaben und sendet sie an `action`.

Attribut	Werte	Funktion
method	get, post	Sendemethode
action	URL (abs/rel)	Sendeziel
name	Bezeichner	Formularname
enctype	multipart/form-data	für File-Upload nötig

FOLIENFEHLER: Auf Day4 steht `<from>` statt `<form>` (Tippfehler).

GET vs. POST

Beide Methoden verschicken Formulardaten, unterscheiden sich aber darin, WO die Daten stehen und wofür man sie einsetzt:

	GET	POST
Übertragung	Parameter in URL ?x=1&y=2	im Body
Sichtbarkeit	in Adresszeile/History sichtbar	nicht in URL
Verwendung	idempotente Abfragen, bookmarkbar	Daten ändern, Upload, sensibel

Weitere Form-Elemente

- `<select>` + `<option>` (`value` , `label` , `selected` , `disabled`); `<optgroup label>` gruppiert.
- `<textarea>` (`cols` , `rows` , `wrap=soft|hard`).
- `<label for="id">` koppelt Beschriftung an Feld-ID (ohne `for` : erstes inneres Feld).
- `<fieldset>` + `<legend>` gruppiert/beschriftet.

Fallen / Bugs

- **Falle:** `<label for>` muss auf die `id` (nicht `name`) zeigen.
- **Falle:** `formaction/formmethod/...` an Buttons überschreiben Form-Werte — laut Folie Zeichen für schlechte Struktur.

Drill

- [Erstellen] Formular, POST an `/save` , ein Textfeld "name", Submit.

Lösung: `<html <form method="post" action="/save"> <input type="text" name="name"> <input type="submit"> </form>`

- [MC] Wo stehen GET-Parameter?

Lösung: in der URL als Query-String.

H5 — Hyperlinks & URLs

- **Definition:** `<a href="...">Text</a>` verweist auf eine Ressource.

<a>-Attribut	Funktion
href	Verweisziel (URL)
target	_self (gleiches Fenster), _blank (neues), Bezeichner (benanntes Fenster)
download	als Download, optional Dateiname
rel	Beziehung: nofollow, noreferrer, bookmark, author, prefetch...
hreflang/type	Sprache / MIME-Typ (nur Vorindikation, Browser liest echten Wert)

URL-Aufbau

Eine URL setzt sich aus festen Bestandteilen zusammen, die jeweils einen Teil des Verweisziels benennen:

`<schema>://<Server>:<Port>/<Pfad>?<Anfrage>#<Fragment>` * **Schema:** http, https, ftp, mailto, file... (Standard-Port: http=80, https=443). * **Query:** `name=wert` Paare, mit `&` verbunden; Sonderzeichen URL-kodieren (`%20` =Leerzeichen). * **Fragment** (`#id`): springt zu Element mit der id; **wird dem Server NIE übermittelt.**

Absolut vs. relativ

Verweise können vollständig (absolut) oder relativ zur aktuellen Seite angegeben werden - das entscheidet über Portabilität:

Form	Bedeutung	Beispiel
absolut	Schema+Host+Pfad	<code>https://x.de/a/b</code>
<code>//host/...</code>	protokoll-relativ (Schema übernommen)	<code>//cdn.x.de/a</code>
<code>/pfad</code>	root-relativ (gleicher Server)	<code>/img/a.png</code>
<code>../, ./, datei.html</code>	dokument-relativ	<code>../a/b.html</code>

Form	Bedeutung	Beispiel
#id	gleiche Ressource, anderes Element	#kap2

Fallen / Bugs

- **MC-Falle:** `/start` ist root-**relativ**, NICHT absolut.
- **Falle:** Fragment `#...` geht nie zum Server (rein clientseitig).
- FOLIE: URL-Kodierung `%uXXXX` für Unicode → **kein gültiger URL-Standard** (legacy JS `escape`); korrekt ist byteweise UTF-8 (`€` = `%E2%82%AC`).

Drill

- [MC] absolut/relativ: `https://x.de`, `/img/a.png`, `../b`, `//cdn/a` ?

Lösung: absolut, root-relativ, dokument-relativ, protokoll-relativ.

H6 — Semantik (optisch vs. semantisch)

optisch (Darstellung)	semantisch (Bedeutung)
<code></code> fett (Stichwort, kein Gewicht)	<code></code> hohe Wichtigkeit/Dringlichkeit
<code><i></code> kursiv	<code></code> Betonung
<code><u></code> unterstrichen	<code><mark></code> markiert

- **Struktur-Tags:** `<header>`, `<main>`, `<footer>`, `<nav>`, `<article>` (eigenständig), `<section>` (themat. Block m. Überschrift), `<aside>`, `<address>` (Kontakt d. Autors).
- **Inline-Semantik:** `<dfn>` Definition, `<cite>` Zitatangabe, `<abbr>` Abkürzung, `<time datetime>` (maschinenlesbar), `<code>`.
- **Block-Semantik:** `<hr>` thematischer Wechsel, `<pre>` vorformatiert (Leerzeichen/Umbrüche erhalten), `<blockquote>` Zitat, `<figure>` + `<figcaption>`.

Fallen / Bugs

- **Kontrast** `b` vs. `strong` / `i` vs. `em`: gleiche Darstellung, aber `strong` / `em` tragen Bedeutung (wichtig für Screenreader/ A11y).
- **Falle:** Semantik nötig für Barrierefreiheit (A11y) — Screenreader nutzt Tags zur Orientierung.

Drill

- [MC] Bedeutung "hohe Wichtigkeit": `<b>` oder `<strong>` ?

Lösung: `<strong>`.

H7 — Tabellen & Listen

Listen

HTML kennt drei Listenarten für unterschiedliche Zwecke:

- `<ul>` unsortiert (Punkt) · `<ol>` sortiert (Zahlen) · `<li>` Element.

Tabellen

Tabellen strukturieren Daten in Zeilen und Spalten; die folgenden Tags bilden das Grundgerüst:

Tag	Funktion
<code><table></code>	Tabelle
<code><tr></code>	Zeile (table row)

`+`

` Aufklappbox (Inhalt bleibt im DOM, nur verborgen). `` (`.show()/modal), `popover`. --- ## Querverweis HTTP (auf Folien im HTML-Teil, gehört zu Säule 4/S2) * GET-Ablauf: Browser → URL (GET) → Server berechnet Seite → HTML zurück → Browser rendert. * HTTP/1.1 (RFC2068, 1997): Request/Response, **zustandslos**, Header + Body. * Statuscodes: 1xx Info · 2xx Erfolg · 3xx Umleitung · 4xx Client-Fehler · 5xx Server-Fehler. FOLIE: kryptische Beschreibungen ("Antwort im ersten Stock") = OCR/Vortrags-Notiz-Müll, ignorieren. → Details in folien-webserver.md.